

Journal Club

23.06.2025

Monte Carlo Basics

Why Monte Carlo?

I want to solve a high dimensional, complicated integral numerically

$$\rightarrow \frac{1}{p^2 - m^2} \text{ peaks}$$

$$\rightarrow \Theta(p_T^{\max} - p_T) \text{ cuts}$$

$\int d\Phi |M|^2$ is typically high dimensional and not well behaved!

$\rightarrow$ Flexible and (somewhat) efficient method:

$$I = \int_{\Omega=[0,1]} dx f(x) \rightarrow \frac{1}{N} \sum_{i=1}^N f(x_i) \equiv I_N$$

where the $\vec{x}_i$ are N random points $x_i \in [0,1]$

$$\hookrightarrow \lim_{N \rightarrow \infty} I_N = I$$

In order for this to be useful in practice, we need to be able to quantify the error:

We calculate the variance of I_N :

$$\text{Var}[I_N] = \dots = \frac{\text{Var}[f(x)]}{N}$$

left as an exercise for the Master students

$$\Rightarrow \delta I_N = \frac{\sqrt{\text{Var}[f(x)]}}{\sqrt{N}} \propto \frac{1}{\sqrt{N}}$$

$\Rightarrow$ Scaling seems horrible, but independent of dimension!

In practice:

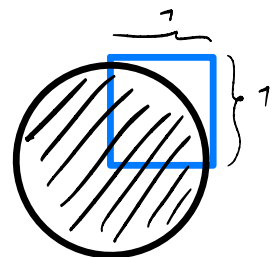
$$\delta I_N = \sqrt{\frac{1}{N} \left[\frac{1}{N} \sum_i f^2(x_i) - \left(\frac{1}{N} \sum_i f(x_i) \right)^2 \right]}$$
$$\text{Var}[f(x)] = E[f^2(x)] - E^2[f(x)]$$

The (un)foreseeable problem:

Imagine you want to integrate the D -dimensional function

$$f_D(x_i) = \Theta\left(1 - \sum_{i=1}^D x_i^2\right)$$

$D=2$ $f_2(x_i) = \Theta(1 - x_1^2 - x_2^2)$



$$\hookrightarrow \int_0^1 dx_1 \int_0^1 dx_2 \Theta(1 - x_1^2 - x_2^2) = \frac{\pi}{4} \sim 0.78$$

$D=20$:

$$I = \int_{[0,1]^D} dx \, \Theta\left(1 - \sum_{i=1}^D x_i^2\right) = \frac{\pi^{\frac{D}{2}}}{2^D \Gamma(\frac{D}{2} + 1)} \sim 2.5 \cdot 10^{-8}$$

↳ Only roughly one in $\frac{1}{I} \sim 4 \cdot 10^7$ points will land in the sphere.

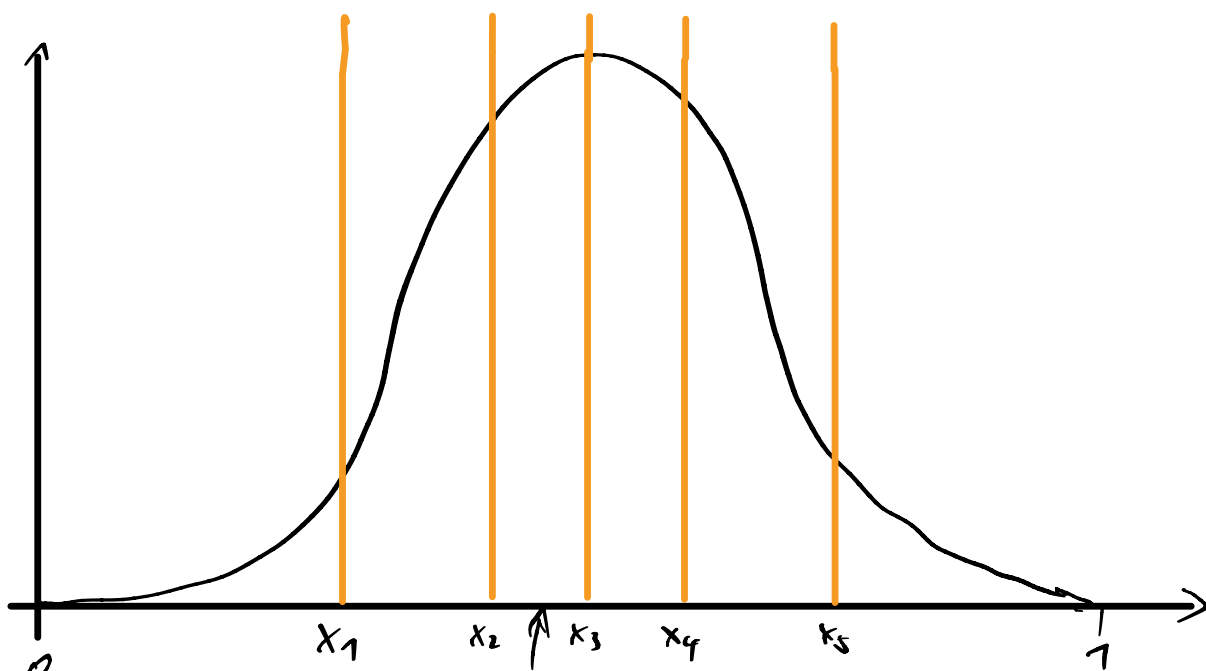
→ Integrand is strongly "peaked"

Naive Monte-Carlo is not enough!

↳ Resort to adaptive sampling.

Adaptive Monte-Carlo:

Put more points where function is "relevant"



Concentrate on this region!

Why does this help?

↳ D=1 example:

$$\int_0^1 dx f(x) = \int_0^1 dx \frac{f(x)}{p(x)} p(x) \equiv E_p\left[\frac{f}{p}\right]$$

$$\sim \frac{1}{N} \sum_{i=1}^N \frac{f(x_i)}{p(x_i)} \quad , \quad x_i \text{ distributed according to } p(x)$$

$$\Rightarrow \text{Var}[I_N] = E_p\left[\frac{f^2}{p^2}\right] - E_p^2\left[\frac{f}{p}\right]$$

↳ left as an exercise for Post-docs ☺
Choose $p \propto |f|$ to minimize the variance

In practice we still want to sample the x_i uniformly:

Introduce $x = h(y)$

$$\Rightarrow \int_0^1 dy \frac{f(x)}{p(x)} p(x) \frac{dx}{dy} = \int_0^1 dy \frac{f(h(y))}{p(h(y))} p(h(y)) h'(y)$$

$$\Rightarrow \text{Choose } h'(y) = \frac{1}{|p(h(y))|} \propto \frac{1}{|f(h(y))|} \quad \hookrightarrow = 1$$

$$\sim \frac{1}{N} \sum_{i=1}^N \frac{f(h(x_i))}{p(h(x_i))} \quad \text{where the } x_i \text{ are distributed uniformly?}$$

The MINT approach:

Avoid $p(x)$ completely:

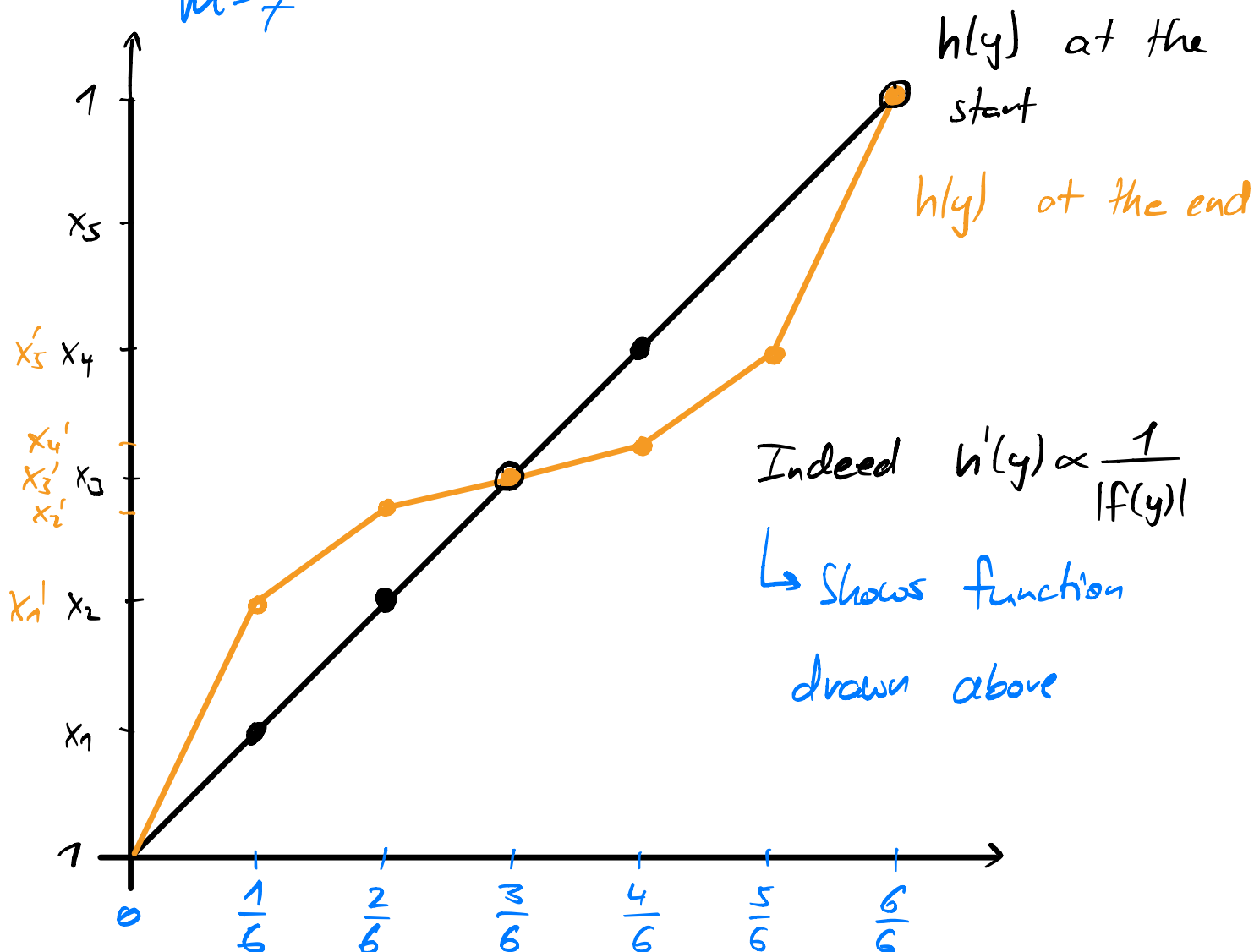
$$\int dx f(x) = \int dy f(h(y)) h'(y)$$

$$\sim \frac{1}{N} \sum_{i=1}^N f(h(x_i)) h'(x_i)$$

and optimize $h'(x_i)$ directly.

→ Define step-wise linear function on a grid

$m=7$



⇒ In POWHEG $m=50$

⇒ Throw points randomly in $[0,1]$:

↳ Calculate contribution to integral and number of hits in each cell

↳ N_L $\Rightarrow$ h'ty optimal if $\frac{R_L}{N_L}$ is constant in L

⇒ Effectively increase N where R is large

↳ Sample proportional to $|f(x)|$

↳ Optimization amounts to solving a set of linear equations

Once the grid is finished:

- Throw 2 random numbers:

r_1 : Picks cell $\in \{1, \dots, 50\}$

r_2 : Picks position in cell

- Because there are more cells where $|f(x)|$ is large this reduces the Monte Carlo error dramatically!

Hands-on session

git clone https://github.com/pat_11BDC2TJQ02jCsQjBGS5XD_UH1jUofFOwpOFAUpE4ETLXOqK1ZgeQlZoRA0Mh1bh7X6PWFDYJYk5jsYRN1@github.com/jakoblinder/MINT.git

[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@github.com/jakoblinder/MINT.git](https://github.com/pat_11BDC2TJQ02jCsQjBGS5XD_UH1jUofFOwpOFAUpE4ETLXOqK1ZgeQlZoRA0Mh1bh7X6PWFDYJYk5jsYRN1@github.com/jakoblinder/MINT.git)

`integrate-functions.f` → Main program doing grid iterations, integration and event generation.

`functions.f` → Defines the functions we want to integrate.

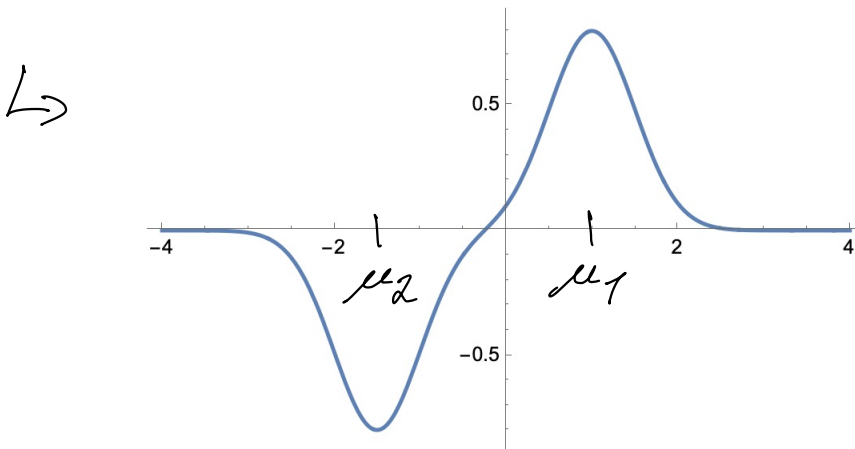
`mint-integrator.f` → Original MINT code with some modifications and extra explanations.

Show some grid plots before looking in the code:

Consider 2 gaussians in 4 dimensions:

$$\rightarrow \left. \begin{array}{l} \mu_1 = +1,5 \\ \mu_2 = -1,5 \end{array} \right\} \sigma_{1/2} = 0,5$$

$$\hookrightarrow f(x_1, \dots, x_4) = \prod_{i=1}^4 g_1(x_i) - \prod_{i=1}^4 g_2(x_i)$$

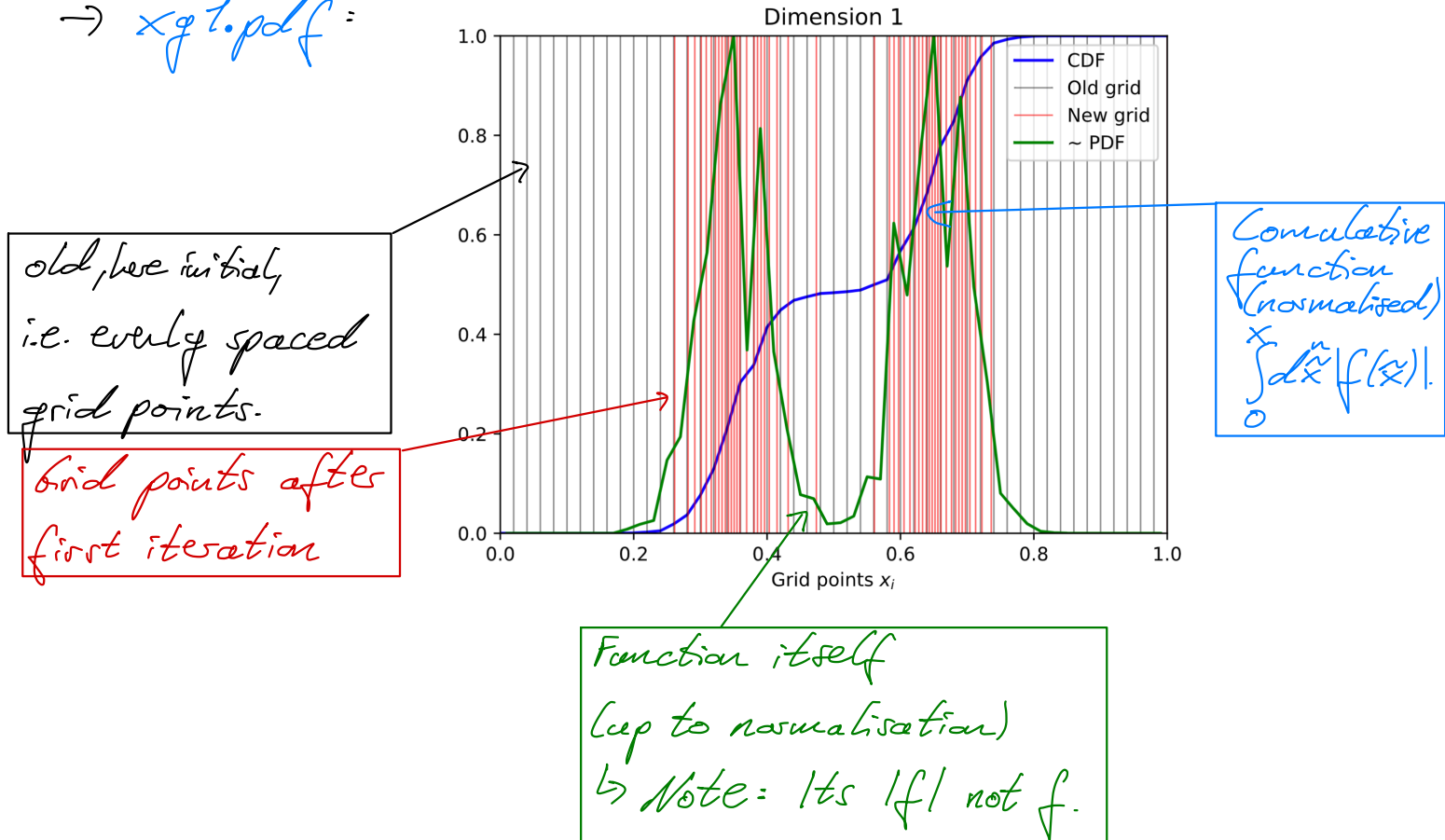


$$\rightarrow \int_{-5}^5 dx^4 |f(x)| = 2 \quad (\text{expected})$$

→ Do $it_{max}=5$ grid iterations with $ncall=10^5$ calls each.

→ Plot the grids: `python plotgrid/plot_topdrawer_grid_useful.py xg?.top`

→ `xg1.pdf`:



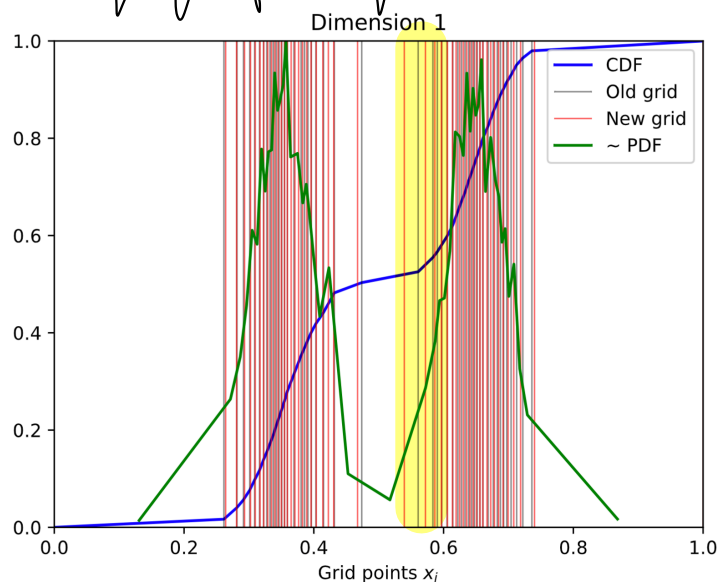
→ Effect of importance sampling:

↳ More grid points where function is big, i.e. CDF is changing fast.

→ Next iteration,

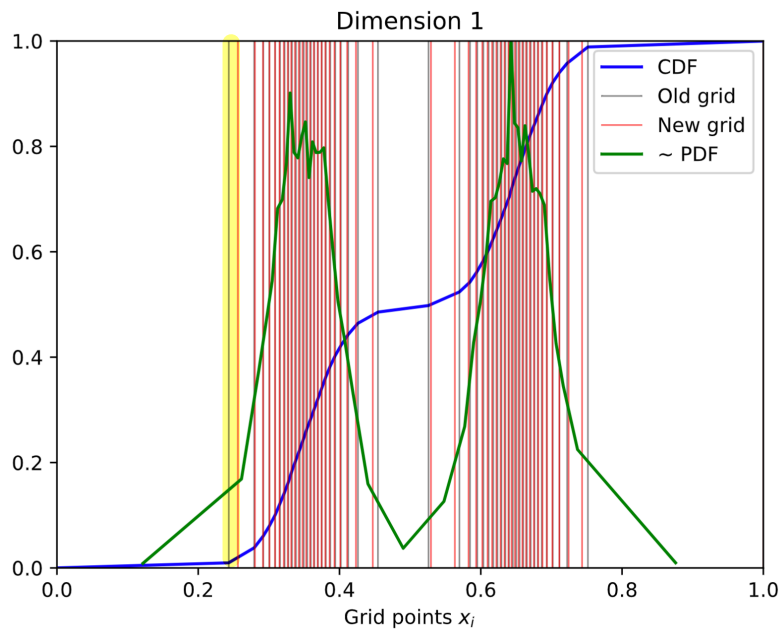
i.e. `xg2.pdf`:

↳ Improvement but way less.



→ 5th iteration,
i.e. $x_{95}.pdf$:

↳ Almost no change.



↳ Integration result: $\int_{-5}^5 dx |f(x)| = 2,013 \pm 0,013$

Considers the code and show how to use:

1.) Fortran:

call mint(func_wrap, ndim, ncall1, itmx1, 0, xgrid, xint, ymax, intabs_val, intabs_err)

↑
Function which is integrated:

↑
Determine grid.

↑
ignore for now ($xint = \int dx |f|$)

↑
outputs

↑
Ignored during grid call.

function func_wrap(x, weight, ifl)

• Demands on the function to be integrated:

function fun(x,w,ifl)
real * 8 fun,w,x(ndim)

When called with ifl=0 it must return $fun=f(x)*w$. When called with ifl=1 it returns $fun=f(x)*w$, but it may assume that, since the previous ifl=0 call the variables that are not folded, and all the values that have been computed with them, have remain unchanged. The return value $f(x)*w$ is not used by mint in this case. When called with ifl=2, fun must return the sum of all the return values up to (and including) the last ifl=0 call.

ifl=0: $fun = f(x) \cdot w$ & $accum = 0$

ifl=0: $fun = f(x) \cdot w$ & $accum += f(x) \cdot w$

ifl=2: $fun = \begin{cases} accum & \text{if } \leq 0 \text{ and } \text{negflag} = \text{true} \\ 0 & \text{false} \end{cases}$

↳ Sets $accum=0$ afterwards.

↳ "w" is needed to take care of the integration volume.

- specify function with a pointer:
 - ↳ set as **10 function** which are exponentiated for simplicity. *show them.*

- x has to be in $[0, 1]$

↳ Add mapping $\bar{x} = x_{\min} + (x_{\max} - x_{\min}) \cdot x$

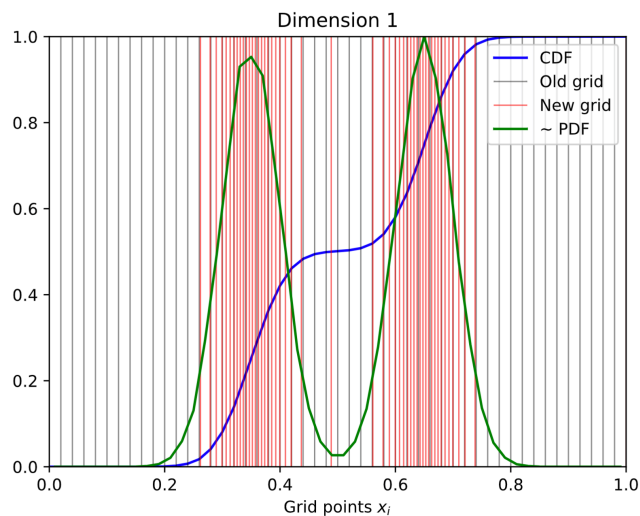
as Rhorry showed in Workshop.

↳ Take care of Jacobian.

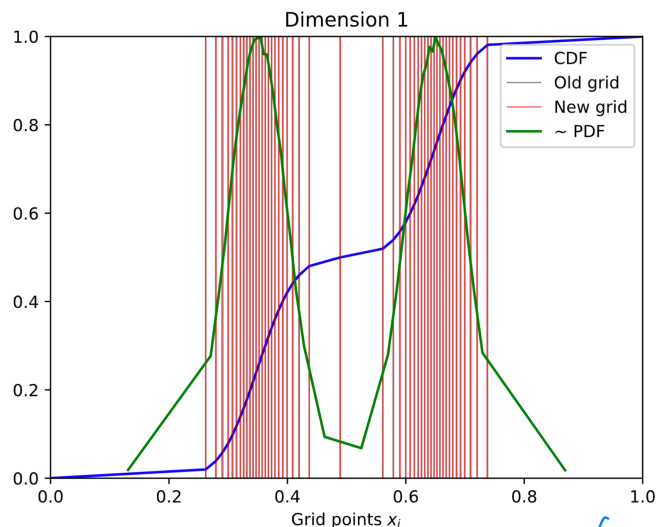
Play a bit around with gauss example:

1.) Increase $n_{\text{call}} = 10^9$:

↳ $x_{\text{gt.pdf}}$ more smooth:



↳ Almost no change in 2nd grid iteration:



$$\Rightarrow \int_{-5}^5 dx |f(x)| = 1,999 \pm 0,001 \quad \left(\frac{1}{\sqrt{100}} \text{ improvement} \right)$$

2.) Use the grid to integrate the gauss ($ncall1=10^6$) $\hookrightarrow$ reduce for speed.

! Compute the positive contribution to the integral:

negflag = .false.

call mint(func_wrap, ndim, ncall2, itmx2, 1, xgrid, xint, ymax, estimp, errorp)

! Compute the negative contribution to the integral:

negflag = .true.

call mint(func_wrap, ndim, ncall2, itmx2, 1, xgrid, xint, ymax, estimn, errorn)

#calls to calculate

MC average of the integral.

Input ∇

Calculate the integral
itmx2 times with ncall2.
 $\hookrightarrow$ Calls and combines the results.

$\rightarrow$ Reduce $ncall1 = 10^6$

$\rightarrow$ Take $ncall2 = 10^6$ and $itmx2 = 5$

$$\hookrightarrow \int_{-5}^5 d^4x f(x) = -0,486 \cdot 10^{-2} \pm 0,347 \cdot 10^{-2}$$

$\hookrightarrow$ Obvious huge (1.0 - 1.0) cancellation!

$\rightarrow$ See the effect of folding $\rightarrow$ ifold = 2 for all dim
($itmx2 = 1$, to be a bit faster)

$\hookrightarrow$ Positive contribution: 0.159818 $\pm$ 0.597918E-03

Negative contribution: -0.158946 $\pm$ 0.820025E-03

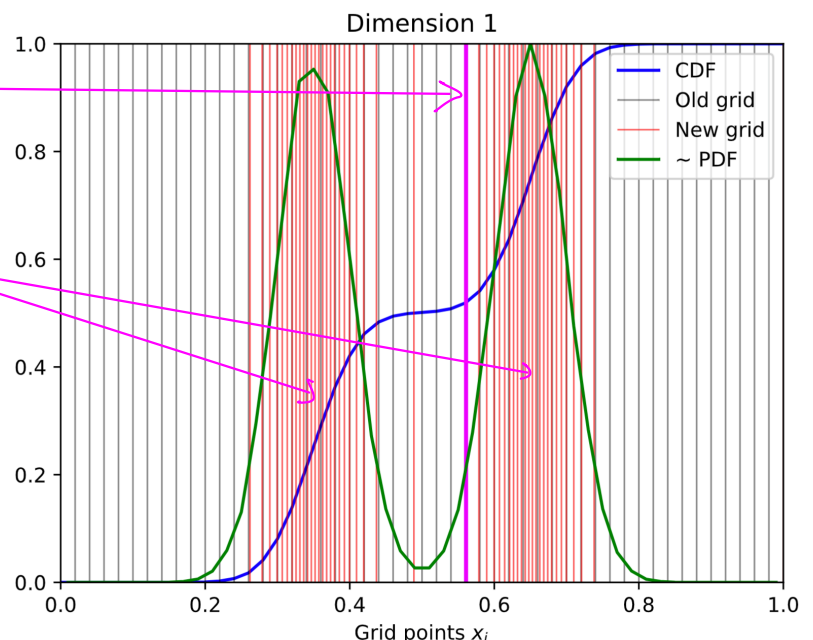
Total integral: 0.872024E-03 $\pm$ 0.101486E-02

Integration time: 16.077 s $\leftarrow 2^4 = 16$ times larger

$$\hookrightarrow \int_{-5}^5 d^4x f(x) = 0,872 \cdot 10^{-3} \pm 0,101 \cdot 10^{-2}$$

Line where folding is done.

Sum up contributions from left and right of the line.



$\hookrightarrow$ python plotgrid/plot_topdrawer_grid_useful.py xg?.top -f 2 2 2 2

↳ Select r_1 -th of the $\frac{m}{ifold}$ cells in each of the areas and sum up:

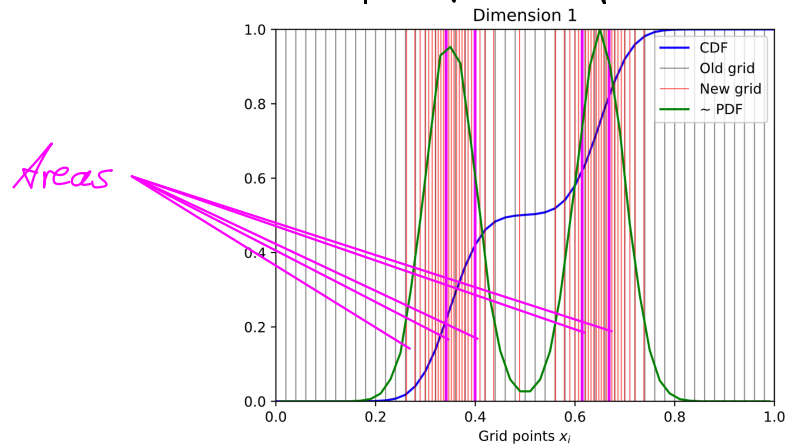
$$\bar{f} = \frac{1}{2} \left(f(x_{r_1} + dx_{r_1} \cdot r_2) + f(x_{1 \cdot \frac{m}{ifold} + r_1} + dx_{1 \cdot \frac{m}{ifold} + r_1} \cdot r_2) \right)$$

for each of the $ncall2$ calls.

↳ Note that the function is called twice now per point.

→ $\prod_{k=1}^{ndim} ifold(k)$ more calls of $f(x)$.

↳ Example grid picture of folding each dimension with 5:



→ General goal of folding: Make a partially negative function positive.

↳ Useful to avoid negative weight events.

→ Show importance of mapping with e^{+x} example:

$$\int_{x_{\min} < 0}^0 d^4x e^{+\sum_i x_i} \stackrel{!}{=} 1$$

↳ Since it's positive, we should see the effect already at grid iteration level, since

$$\int_{x_{\min}}^0 d^4x |e^{+\sum_i x_i}| \text{ is returned there.}$$

↳ $n_{\text{calls}} = 2 = 10^6$ & $it_{\text{max}} = 1$ works well for $x_{\min} = -10^4$.

↳ $x_{\min} = -10^5$ does not work. Since e^{+x} goes to zero too fast ($e^{-100} \sim 10^{-44}$), we are getting a divide by zero error somewhere in the grid iteration routine.

↳ Using a logarithmic mapping solves that:

$$\tilde{x} = \ln [e^{x_{\min}} + (e^{x_{\max}} - e^{x_{\min}}) \cdot x]$$

→ Need jacobian.

Short comment on the plots:

(analysis) linder@Jakobs-MacBook-Pro MINT % python plotgrid/plot_topdrawer_grid_useful.py -h

usage: plot_topdrawer_grid_useful.py [-h] [-f [FOLDING ...]] input [input ...]

Plot MINT topdrawer grid file to PDF (one page per dimension).

positional arguments:

input Input topdrawer file(s) for grid, supports glob patterns (e.g. pwg-xg2-xgrid-btl-*.top).

options:

-h, --help show this help message and exit

-f [FOLDING ...], --folding [FOLDING ...]

Optional list of numbers (one per dimension or less), visualising which blocks MINT is adding during the folding. Note that the folding numbers are specified in reverse order, i.e. the first entry is the folding factor of the last dimension, the second entry of the second to last dimension, etc..If not specified, no folding is visualised.

→ Use for your POWHEG grids.